ЛЕКЦИЯ 5. FLUTTER: ОСНОВЫ И ВИДЖЕТЫ

FLUTTER: ЧТО ЭТО?

- Фреймворк от **Google** (2018) для кроссплатформенной разработки
- Язык: **Dart**
- Собственный рендеринг: **Impeller** (замена Skia с 2024)
- Платформы: Android, iOS, Web, Windows, macOS, Linux
- Одна кодовая база - 6 платформ
- Полное нативное покрытие API обеих мобильных платформ

FLUTTER: АРХИТЕКТУРА

Framework (Dart)

Material Design, Cupertino Widgets, Animation, Gestures

Engine (C++)

Impeller/Skia, Dart Runtime, Platform Channels, Text Layout

Embedder

Связь с ОС: Android (NDK), iOS, Desktop

FLUTTER: **ПРЕИМУЩЕСТВА**

- **Hot Reload** - мгновенное обновление UI без перезапуска
- **Pixel-perfect** - одинаковый вид на всех устройствах
- **Нативная производительность** - компиляция в нативный код
- **Богатая экосистема** - 40000+ пакетов (pub.dev)
- **Multiplatform** - не только мобильные платформы

ЯЗЫК DART

DART: ОСНОВЫ

```
// Переменные
var name = 'Flutter';       // тип выводится автоматически
String lang = 'Dart';       // явный тип
final pi = 3.14;            // неизменяемая после инициализации
const maxAge = 150;          // константа времени компиляции

// Функции
int add(int a, int b) => a + b;  // стрелочная функция

// Строка с интерполяцией
print('Язык: $name, версия: ${add(2, 3)}');

// Null safety
String? nullableName;            // может быть null
String nonNullName = 'Hello';  // не может быть null
print(nullableName?.length ?? 0);  // null-aware operator
```

DART: ИМЕНОВАННЫЕ И ОПЦИОНАЛЬНЫЕ ПАРАМЕТРЫ

```
// Именованные параметры (required и optional)
void createUser({
  required String name,
  int age = 0,
  String? email,
}) {
  print('$name, $age, $email');
}

createUser(name: 'Alice', age: 25);
createUser(name: 'Bob');  // email = null, age = 0

// Позиционные опциональные параметры
String repeat(String text, [int times = 2]) {
  return text * times;
}
```

```
print(repeat('Ha'));        // HaHa
print(repeat('Ha', 3));    // HaHaHa

// Разделение именованных и позиционных
void greet(String greeting, {String? name}) {
  print('$greeting, $name!');
}
```

DART: FACTORY КОНСТРУКТОРЫ И АБСТРАКТНЫЕ КЛАССЫ

```
// Factory constructor - может вернуть существующий экземпляр
class Logger {
  static final Map<String, Logger> _cache = {};

  factory Logger(String name) {
    return _cache.putIfAbsent(name, () => Logger._internal(name));
  }

  Logger._internal(this.name);
  final String name;
}

// Абстрактный класс
abstract class Shape {
  double get area;
  void describe() => print('Area: $area');
}
```

```
class Circle extends Shape {
  Circle(this.radius);
  final double radius;
  @override
  double get area => 3.14159 * radius * radius;
}

class Rectangle extends Shape {
  Rectangle(this.width, this.height);
  final double width, height;
  @override
  double get area => width * height;
}
```

DART: ENUM С МЕТОДАМИ

```
// Enum с полями и методами
enum Color {
  red('FF0000', 'Красный'),
  green('00FF00', 'Зелёный'),
  blue('0000FF', 'Синий');

  const Color(this.hex, this.label);
  final String hex;
  final String label;

  bool get isPrimary =>
      hex == 'FF0000' || hex == '00FF00' || hex == '0000FF';
}

// Enhance enums (Dart 3+)
enum HTTPStatus {
  ok(200, 'OK'),
  notFound(404, 'Not Found'),
  serverError(500, 'Server Error');
```

```
  const HTTPStatus(this.code, this.message);
  final int code;
  final String message;

  bool get isSuccess => code >= 200 && code < 300;
}

print(HTTPStatus.ok.label);         // OK
print(HTTPStatus.notFound.code);    // 404
```

DART: PATTERN MATCHING (SWITCH EXPRESSIONS)

```
// Switch expression (Dart 3+)
String describeAnimal(String type) => switch (type) {
  'cat' => 'Кошка',
  'dog' => 'Собака',
  'bird' => 'Птица',
  _ => 'Неизвестное животное',
};

// Pattern matching с Records
(String, int) parseUser(String input) {
  final parts = input.split(':');
  return (parts[0], int.parse(parts[1]));
}

var (name, age) = parseUser('Alice:30');

// Guard patterns
```

```
String classify(int value) => switch (value) {
  > 0 => 'Положительное',
  0 => 'Ноль',
  < 0 => 'Отрицательное',
};

// Destructuring
const point = (x: 10, y: 20);
final (x: px, y: py) = point;
print('x=$px, y=$py');
```

DART: КОЛЛЕКЦИИ

```
// List literal
var fruits = ['яблоко', 'банан', 'апельсин'];

// Set literal (уникальные элементы)
var colors = {'красный', 'зелёный', 'синий'};

// Map literal
var capitals = {
  'Россия': 'Москва',
  'Франция': 'Париж',
  'Япония': 'Токио',
};

// Spread operator
var extended = [...fruits, 'киви'];
var merged = {...colors, 'жёлтый'};

// Null-aware spread
List<String>? nullableList;
```

```
var safe = [...?nullableList, 'дефолт'];

// Collection if/for
var commands = [
  'SELECT * FROM users',
  if (true) 'WHERE active = 1',
  for (var i = 0; i < 3; i++) 'LIMIT $i',
];
```

DART: NULL SAFETY ОПЕРАТОРЫ (?. ?? !)

```
String? name;
int? length;

// ?. - Null-aware access
length = name?.length;  // null если name == null

// ?? - Null-aware fallback (エルビス演算子)
var displayName = name ?? 'Аноним';

// ??= - Null-aware assignment
name ??= 'Дефолт';

// ! - Non-null assertion (опасно!)
var forcedLength = name!.length;  // AssertionError если null

// ?.. - Null-aware cascade
name?.toLowerCase()?.trim();
```

```
// ?? () - Null-aware function call
String? Function()? getGreeting;
var result = getGreeting?.() ?? 'Привет!';

// Практический пример
class User {
  User({this.name, this.address});
  String? name;
  String? address;
}

var user = User();
var city = user.address?.city ?? 'Неизвестно';
```

DART: АСИНХРОННОСТЬ

```
// Future - асинхронный результат
Future<String> fetchData() async {
  final response = await http.get(Uri.parse('https://api.examp
  return response.body;
}

// Stream - последовательность асинхронных событий
Stream<int> countStream() async* {
  for (int i = 0; i < 5; i++) {
    await Future.delayed(Duration(seconds: 1));
    yield i;
  }
}

// Использование
await for (final value in countStream()) {
  print(value);  // 0, 1, 2, 3, 4
}
```

DART: ASYNC GENERATORS

```
// Async generator - возвращает Stream, использует yield
Stream<int> numbersFromDB() async* {
  var page = 0;
  while (true) {
    var data = await fetchPage(page);
    if (data.isEmpty) break;
    for (var item in data) {
      yield item;  // отдаём значение в Stream
    }
    page++;
  }
}

// yield* - делегирование другого Stream
Stream<String> allNames() async* {
  yield* fetchAdminNames();
  yield* fetchUserNames();
}
```

```
// Интерактивный async generator
Stream<String> watchFile(String path) async* {
  await for (var change in File(path).watch()) {
    yield 'Изменение: ${change.type}';
  }
}

// Использование
await for (var n in numbersFromDB().take(10)) {
  print(n);
}
```

DART: AOT VS JIT

JIT (DEBUG MODE)

- Компиляция "на лету"
- Используется при разработке
- Поддерживает Hot Reload
- Медленнее при запуске

AOT (RELEASE MODE)

- Компиляция перед запуском
- Используется в production
- Быстрый запуск и работа
- Оптимизированный код

ВИДЖЕТЫ (WIDGETS)

ВСЁ - ВИДЖЕТЫ

- "Everything is a widget" - философия Flutter
- Виджет - описание части UI
- Виджеты образуют **Widget Tree**
- Каждый виджет - immutable (неизменяемый)
- Два типа: **StatelessWidget** и **StatefulWidget**

STATELESSWIDGET

```
class Greeting extends StatelessWidget {
  final String name;

  const Greeting({super.key, required this.name});

  @override
  Widget build(BuildContext context) {
    return Text(
      'Привет, $name!',
      style: const TextStyle(fontSize: 24),
    );
  }
}

// Использование
Greeting(name: 'Мир')
```

StatelessWidget: immutable, не имеет внутреннего состояния.

STATEFULWIDGET

```
class Counter extends StatefulWidget {
  const Counter({super.key});

  @override
  State<Counter> createState() => _CounterState();
}

class _CounterState extends State<Counter> {
  int _count = 0;

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Счётчик: $_count'),
        ElevatedButton(
          onPressed: () {
            setState(() {
              _count++;
            });
```

```
            },
            child: const Text('+1'),
          ),
        ],
      );
  }
}
```

КЛЮЧЕВЫЕ ВСТРОЕННЫЕ ВИДЖЕТЫ

ОСНОВНЫЕ

- **Text** - текст
- **Image** - изображение
- **Icon** - иконка
- **ElevatedButton** - кнопка
- **TextField** - поле ввода

КОНТЕЙНЕРЫ

- **Container** - универсальный контейнер
- **Padding** - отступ
- **Center** - центрирование
- **Expanded** - заполнение пространства
- **SizedBox** - фиксированный размер

GLOBALKEY

```
// GlobalKey - уникальный ключ, доступ к State из любого места
final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
final GlobalKey<_MyWidgetState> myKey = GlobalKey();

// Доступ к State через GlobalKey
class MyWidget extends StatefulWidget {
  const MyWidget({super.key});
  @override
  State<MyWidget> createState() => _MyWidgetState();
}

class _MyWidgetState extends State<MyWidget> {
  String data = 'Hello';
  @override
  Widget build(BuildContext context) => Text(data);
}

// Использование:
// myKey.currentState?.data = 'World';
```

```
// myKey.currentState?.setState(() {});

// GlobalKey для Form
Form(
  key: _formKey,
  child: TextFormField(validator: (v) => v!.isEmpty ? 'Error'
)

// Валидация формы из любого места
_formKey.currentState?.validate();
```

INHERITEDWIDGET

```
// InheritedWidget - передача данных вниз по дереву виджетов
class ThemeProvider extends InheritedWidget {
  const ThemeProvider({
    super.key,
    required this.isDark,
    required super.child,
  });

  final bool isDark;

  static ThemeProvider? of(BuildContext context) {
    return context.dependOnInheritedWidgetOfExactType<ThemePro
  }

  @override
  bool updateShouldNotify(ThemeProvider oldWidget) =>
      isDark != oldWidget.isDark;
}
```

```
// Использование
class App extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return ThemeProvider(
      isDark: true,
      child: Builder(
        builder: (context) {
          final isDark = ThemeProvider.of(context)?.isDark ??
          return MaterialApp(
            theme: isDark ? ThemeData.dark() : ThemeData.light
          );
        },
      ),
    );
  }
}
```

THEME И MEDIAQUERY

```
// Доступ к текущей теме
final theme = Theme.of(context);
final primaryColor = theme.colorScheme.primary;

// Доступ к информации об устройстве
final size = MediaQuery.of(context).size;
final width = size.width;
final height = size.height;
final padding = MediaQuery.of(context).padding;
final orientation = MediaQuery.of(context).orientation;

// Пример: безопасная область
Scaffold(
  body: Padding(
    padding: EdgeInsets.only(top: MediaQuery.of(context).paddi
    child: Text('Under status bar'),
  ),
)
```

```
// Изменение размера шрифта (масштабирование)
Text(
  'Масштабируемый текст',
  style: TextStyle(fontSize: 16 * MediaQuery.of(context).textS
)
```

LAYOUTBUILDER И ORIENTATIONBUILDER

```
// LayoutBuilder - адаптивный layout на основе доступного прос
LayoutBuilder(
  builder: (context, constraints) {
    if (constraints.maxWidth > 600) {
      return Row(children: [_buildSidebar(), _buildContent()])
    } else {
      return _buildContent();
    }
  },
)

// OrientationBuilder - реагирует на смену ориентации
OrientationBuilder(
  builder: (context, orientation) {
    return GridView.builder(
      gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: orientation == Orientation.portrait ?
```

```
      ),
      itemBuilder: (context, index) => Card(child: Center(chil
    );
  },
)
```

ASPECTRATIO И FRACTIONALLYSIZEDBOX

```
// AspectRatio - сохраняет пропорции
AspectRatio(
  aspectRatio: 16 / 9,
  child: Container(
    color: Colors.blue,
    child: const Center(child: Text('16:9')),
  ),
)

// FractionallySizedBox - доля от родителя
FractionallySizedBox(
  widthFactor: 0.8,   // 80% ширины родителя
  heightFactor: 0.3,  // 30% высоты родителя
  alignment: Alignment.center,
  child: Container(
    color: Colors.green,
    child: const Text('80% x 30%'),
```

```
  ),
)

// Пример: адаптивная карточка
FractionallySizedBox(
  widthFactor: MediaQuery.of(context).size.width > 600 ? 0.5 :
  child: AspectRatio(
    aspectRatio: 3 / 2,
    child: Card(child: _buildContent()),
  ),
)
```

CONTAINER: УНИВЕРСАЛЬНЫЙ ВИДЖЕТ

```
Container(
  width: 200,
  height: 100,
  padding: const EdgeInsets.all(16),
  margin: const EdgeInsets.symmetric(horizontal: 20),
  decoration: BoxDecoration(
    color: Colors.blue,
    borderRadius: BorderRadius.circular(12),
    boxShadow: [
      BoxShadow(
        color: Colors.black26,
        blurRadius: 8,
        offset: Offset(0, 4),
      ),
    ],
  ),
  child: const Text(
```

```
    'Привет!',
    style: TextStyle(color: Colors.white, fontSize: 18),
  ),
)
```

LAYOUT: ROW И COLUMN

```
// Горизонтальное расположение
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: [
    Icon(Icons.star, color: Colors.yellow),
    Icon(Icons.star, color: Colors.yellow),
    Icon(Icons.star, color: Colors.yellow),
  ],
)

// Вертикальное расположение
Column(
  crossAxisAlignment: CrossAxisAlignment.stretch,
  children: [
    Text('Заголовок', style: TextStyle(fontSize: 24)),
    SizedBox(height: 8),
    Text('Подзаголовок'),
  ],
```

)

LAYOUT: СПИСКИ

```
// ListView - прокручиваемый список
ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, index) {
    return ListTile(
      leading: Icon(Icons.person),
      title: Text(items[index].name),
      subtitle: Text(items[index].email),
      trailing: Icon(Icons.chevron_right),
      onTap: () => print('Нажали: ${items[index].name}'),
    );
  },
)

// GridView - сетка
GridView.builder(
  gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,
    crossAxisSpacing: 8,
```

```
    mainAxisSpacing: 8,
  ),
  itemBuilder: (context, index) => Card(child: Center(child: T
)
```

SLIVERS: SLIVERLIST И SLIVERGRID

```
// CustomScrollView с Sliver'ами
CustomScrollView(
  slivers: [
    SliverAppBar(
      expandedHeight: 200,
      floating: true,
      pinned: true,
      flexibleSpace: FlexibleSpaceBar(
        title: const Text('Заголовок'),
        background: Image.network('url', fit: BoxFit.cover),
      ),
    ),
    SliverList(
      delegate: SliverChildBuilderDelegate(
        (context, index) => ListTile(title: Text('Элемент $index')),
        childCount: 20,
      ),
    ),
    SliverGrid(
      delegate: SliverChildBuilderDelegate(
```

```
        (context, index) => Card(child: Center(child: Text('Сетка $
        childCount: 12,
      ),
      gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount
        crossAxisCount: 3,
      ),
    ),
  ],
)
```

CUSTOMSCROLLVIEW И SHRINKWRAPPING LISTVIEW

```
// ShrinkWrapping ListView - встраивается в родителя
ListView(
  shrinkWrap: true,  //重要!
  physics: NeverScrollableScrollPhysics(),
  children: [
    ListTile(title: Text('Элемент 1')),
    ListTile(title: Text('Элемент 2')),
    ListTile(title: Text('Элемент 3')),
  ],
)

// NestedScrollView - вложенные списки
NestedScrollView(
  headerSliverBuilder: (context, innerBoxIsScrolled) => [
    SliverAppBar(title: const Text('Заголовок'), floating: true),
  ],
  body: ListView.builder(
    itemCount: 50,
```

```
    itemBuilder: (context, index) => ListTile(title: Text('Элемент
  ),
)
```

MATERIAL DESIGN И CUPERTINO

MATERIAL DESIGN (ANDROID)

```
MaterialApp(
  title: 'Моё приложение',
  theme: ThemeData(
    colorSchemeSeed: Colors.blue,
    useMaterial3: true,
  ),
  home: Scaffold(
    appBar: AppBar(
      title: const Text('Главная'),
      actions: [
        IconButton(
          icon: const Icon(Icons.search),
          onPressed: () {},
        ),
      ],
    ),
    body: const Center(child: Text('Hello!')),
    floatingActionButton: FloatingActionButton(
      onPressed: () {},
```

```
      child: const Icon(Icons.add),
    ),
  ),
)
```

CUPERTINO (IOS-СТИЛЬ)

```
CupertinoApp(
  title: 'Моё приложение',
  theme: const CupertinoThemeData(
    primarySwatch: Colors.blue,
  ),
  home: CupertinoPageScaffold(
    navigationBar: const CupertinoNavigationBar(
      middle: Text('Главная'),
    ),
    child: const Center(
      child: Text('Hello, iOS!'),
    ),
  ),
)
```

АДАПТИВНЫЙ UI

```
// Определение платформы
import 'dart:io';

Widget build(BuildContext context) {
  if (Platform.isIOS) {
    return CupertinoPageScaffold(
      navigationBar: const CupertinoNavigationBar(middle: Text
      child: _buildContent(),
    );
  } else {
    return Scaffold(
      appBar: AppBar(title: const Text('App')),
      body: _buildContent(),
    );
  }
}

// Или использовать Platform Widgets
```

```
// https://pub.dev/packages/flutter_platform_widgets
```

ФОРМЫ: FORM И TEXTFORMFIELD

```
class RegistrationForm extends StatefulWidget {
  @override
  State<RegistrationForm> createState() => _RegistrationFormSt
}

class _RegistrationFormState extends State<RegistrationForm> {
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();

  @override
  Widget build(BuildContext context) {
    return Form(
      key: _formKey,
      child: Column(
        children: [
          TextFormField(
            controller: _emailController,
            decoration: const InputDecoration(
```

```
        labelText: 'Email',
        hintText: 'user@example.com',
        prefixIcon: Icon(Icons.email),
        border: OutlineInputBorder(),
      ),
    ),
    TextFormField(
      controller: _passwordController,
      obscureText: true,
      decoration: const InputDecoration(
        labelText: 'Пароль',
        prefixIcon: Icon(Icons.lock),
      ),
    ),
    ElevatedButton(
      onPressed: () {
        if (_formKey.currentState!.validate()) {
          print('Форма валидна!');
        }
      },
      child: const Text('Отправить'),
    ),
```

```
          ],
        ),
      );
    }
}
```

ФОРМЫ: ВАЛИДАЦИЯ И INPUTDECORATION

```
// Валидаторы
TextFormField(
  validator: (value) {
    if (value == null || value.isEmpty) {
      return 'Обязательное поле';
    }
    if (!RegExp(r'^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$').hasMatch(value
      return 'Некорректный email';
    }
    return null;  // null = валидно
  },
)

// InputDecoration - настройка внешнего вида
InputDecoration(
  labelText: 'Имя',
  labelStyle: TextStyle(color: Colors.grey),
  hintText: 'Введите имя',
```

```
  hintStyle: TextStyle(color: Colors.grey.shade300),
  prefixIcon: Icon(Icons.person, color: Colors.blue),
  suffixIcon: IconButton(
    icon: Icon(Icons.clear),
    onPressed: () => _controller.clear(),
  ),
  border: OutlineInputBorder(
    borderRadius: BorderRadius.circular(10),
  ),
  filled: true,
  fillColor: Colors.grey.shade50,
)
```

ЖЕСТЫ: GESTUREDETECTOR И INKWELL

```
// GestureDetector - обработка жестов
GestureDetector(
  onTap: () => print('Tap'),
  onDoubleTap: () => print('Double tap'),
  onLongPress: () => print('Long press'),
  onPanUpdate: (details) => print('Delta: ${details.delta}'),
  child: Container(
    width: 200,
    height: 100,
    color: Colors.blue,
    child: const Center(child: Text('Нажми меня')),
  ),
)

// InkWell - с эффектом涟漪 (Material ripple)
InkWell(
  onTap: () => print('Нажали!'),
```

```
  onLongPress: () => print('Долгое нажатие'),
  borderRadius: BorderRadius.circular(10),
  splashColor: Colors.blue.withOpacity(0.3),
  child: Container(
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      border: Border.all(color: Colors.blue),
      borderRadius: BorderRadius.circular(10),
    ),
    child: const Text('Нажми меня'),
  ),
)
```

ЖЕСТЫ: DISMISSIBLE И DRAGTARGET

```
// Dismissible - свайп для удаления
Dismissible(
  key: Key(item.id),
  direction: DismissDirection.endToStart,
  onDismissed: (direction) => removeItem(item),
  background: Container(
    color: Colors.red,
    alignment: Alignment.centerRight,
    padding: const EdgeInsets.only(right: 20),
    child: const Icon(Icons.delete, color: Colors.white),
  ),
  child: ListTile(title: Text(item.name)),
)

// LongPressDraggable + DragTarget
LongPressDraggable(
  data: item,
  feedback: Material(child: Text(item.name)),
  childWhenDragging: Opacity(
    opacity: 0.5,
```

```
    child: ListTile(title: Text(item.name)),
  ),
  child: ListTile(title: Text(item.name)),
)

DragTarget(
  onAcceptWithDetails: (data) => addItem(data),
  builder: (context, candidateData, rejectedData) => Container(
    height: 100,
    color: candidateData.isNotEmpty ? Colors.green : Colors.grey,
    child: const Center(child: Text('Перетащи сюда')),
  ),
)
```

ИЗОБРАЖЕНИЯ: IMAGE.NETWORK, IMAGE.FILE, IMAGE.MEMORY

```
// Из сети
Image.network(
  'https://picsum.photos/400/300',
  width: 400,
  height: 300,
  fit: BoxFit.cover,
  loadingBuilder: (context, child, loadingProgress) {
    if (loadingProgress == null) return child;
    return CircularProgressIndicator();
  },
  errorBuilder: (context, error, stackTrace) => Icon(Icons.error),
)

// Из файла
Image.file(
  File('/path/to/image.jpg'),
  width: 200,
  height: 200,
```

```
  fit: BoxFit.contain,
)

// Из памяти (байты)
Image.memory(
  bytes,  // Uint8List
  width: 100,
  height: 100,
)
```

ИЗОБРАЖЕНИЯ: FADEINIMAGE И CACHEDNETWORKIMAGE

```
// FadeInImage - плавное появление изображения
FadeInImage(
  placeholder: AssetImage('assets/placeholder.png'),
  image: NetworkImage('https://picsum.photos/400/300'),
  width: 400,
  height: 300,
  fit: BoxFit.cover,
  fadeInDuration: const Duration(milliseconds: 300),
  fadeOutDuration: const Duration(milliseconds: 100),
)

// CachedNetworkImage - кэширование из сети
// pub.dev: cached_network_image
CachedNetworkImage(
  imageUrl: 'https://picsum.photos/400/300',
  placeholder: (context, url) =>
      const Center(child: CircularProgressIndicator()),
  errorWidget: (context, url, error) =>
```

```
      const Icon(Icons.error),
  width: 400,
  height: 300,
  fit: BoxFit.cover,
  memCacheWidth: 400,  // оптимизация памяти
)
```

ТИПОГРАФИКА И СТИЛИЗАЦИЯ ТЕКСТА

```
Text(
  'Заголовок',
  style: TextStyle(
    fontSize: 32,
    fontWeight: FontWeight.bold,
    fontFamily: 'Roboto',
    color: Colors.blue.shade800,
    letterSpacing: 1.5,
    wordSpacing: 3.0,
    height: 1.5,
    foreground: Paint()
      ..shader = LinearGradient(
        colors: [Colors.red, Colors.blue],
      ).createShader(Rect.fromLTWH(0, 0, 200, 30)),
  ),
)
```

```
// TextTheme из темы
final theme = Theme.of(context);
Text('Display Large', style: theme.textTheme.displayLarge);
Text('Title Medium', style: theme.textTheme.titleMedium);
Text('Body Small', style: theme.textTheme.bodySmall);

// Google Fonts (пакет)
Text('Шрифт из Google', style: GoogleFonts.roboto(fontSize: 24
```

RICHTEXT И TEXTSPAN

```
RichText(
  text: TextSpan(
    style: const TextStyle(color: Colors.black, fontSize: 16),
    children: [
      const TextSpan(text: 'Обычный текст '),
      TextSpan(
        text: 'жирный',
        style: const TextStyle(fontWeight: FontWeight.bold),
      ),
      const TextSpan(text: ' и '),
      TextSpan(
        text: 'ссылка',
        style: const TextStyle(
          color: Colors.blue,
          decoration: TextDecoration.underline,
        ),
        recognizer: TapGestureRecognizer()..onTap = () => print('Ta
      ),
    ],
  ),
```

```
)

// SelectableText - выделяемый текст
SelectableText.rich(
  TextSpan(
    children: [
      TextSpan(text: 'Email: ', style: TextStyle(fontWeight: FontWe
      TextSpan(text: 'user@example.com', style: TextStyle(color: Co
    ],
  ),
)
```

ПЕРВОЕ ПРИЛОЖЕНИЕ НА FLUTTER

СОЗДАНИЕ ПРОЕКТА

```
# Установка Flutter SDK
# https://docs.flutter.dev/get-started/install

# Проверка установки
flutter doctor

# Создание проекта
flutter create my_app
cd my_app

# Запуск
flutter run

# Сборка APK (Android)
flutter build apk

# Сборка IPA (iOS)
```

```
flutter build ios
```

PUBSPEC.YAML: КОНФИГУРАЦИЯ

```
name: my_app
description: Моё первое Flutter-приложение
version: 1.0.0+1

environment:
  sdk: '>=3.2.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.0
  provider: ^6.1.1
  shared_preferences: ^2.2.2

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^4.0.0

flutter:
```

```
uses-material-design: true
assets:
  - assets/images/
```

MAIN.DART: ТОЧКА ВХОДА

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Моё приложение',
      theme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        useMaterial3: true,
      ),
      home: const HomeScreen(),
    );
```

```
  }
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return const Scaffold(
      body: Center(
        child: Text('Привет, Flutter!'),
      ),
    );
  }
}
```

КАСТОМИЗАЦИЯ ТЕМЫ: COLORSCHEME И TEXTTHEME

```
ThemeData(
  colorSchemeSeed: Colors.deepPurple,
  brightness: Brightness.light,
  useMaterial3: true,
  textTheme: const TextTheme(
    displayLarge: TextStyle(fontSize: 57, fontWeight: FontWeight.w4
    titleLarge: TextStyle(fontSize: 22, fontWeight: FontWeight.w500
    bodyMedium: TextStyle(fontSize: 14, fontWeight: FontWeight.w400
  ),
  appBarTheme: const AppBarTheme(
    centerTitle: true,
    elevation: 0,
  ),
  cardTheme: CardTheme(
    elevation: 2,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circul
  ),
  elevatedButtonTheme: ElevatedButtonThemeData(
```

```
    style: ElevatedButton.styleFrom(
      padding: const EdgeInsets.symmetric(horizontal: 24, vertical:
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circ
    ),
  ),
)
```

SHAPEDECORATION И ДЕКОРАЦИИ

```
// BoxDecoration -最常见的
Container(
  decoration: BoxDecoration(
    gradient: const LinearGradient(
      begin: Alignment.topLeft,
      end: Alignment.bottomRight,
      colors: [Colors.blue, Colors.purple],
    ),
    borderRadius: BorderRadius.circular(16),
    border: Border.all(color: Colors.white, width: 2),
    boxShadow: [
      BoxShadow(color: Colors.black26, blurRadius: 8, offset:
    ],
  ),
)

// ShapeDecoration
Container(
  width: 100,
```

```
  height: 100,
  decoration: ShapeDecoration(
    shape: StarBorder(),
    gradient: LinearGradient(colors: [Colors.amber, Colors.ora
    shadows: [
      BoxShadow(color: Colors.orange.shade300, blurRadius: 10)
    ],
  ),
)

// ClipRRect + Image
ClipRRect(
  borderRadius: BorderRadius.circular(50),
  child: Image.network('url', width: 100, height: 100),
)
```

CUSTOMPAINT И CUSTOMPAINTER

```
// CustomPaint - рисование на Canvas
class MyPainter extends CustomPainter {
  @override
  void paint(Canvas canvas, Size size) {
    final paint = Paint()
      ..color = Colors.blue
      ..strokeWidth = 3
      ..style = PaintingStyle.stroke;

    // Линия
    canvas.drawLine(Offset(0, 0), Offset(size.width, size.height),

    // Круг
    canvas.drawCircle(
      Offset(size.width / 2, size.height / 2),
      50,
      paint..style = PaintingStyle.fill,
    );

    // Прямоугольник
```

```
    canvas.drawRect(
      Rect.fromLTWH(10, 10, 100, 50),
      paint..color = Colors.red,
    );
  }

  @override
  bool shouldRepaint(MyPainter oldDelegate) => false;
}

// Использование
CustomPaint(
  size: Size(300, 300),
  painter: MyPainter(),
)
```

CANVAS API: ОСНОВЫ

```
@override
void paint(Canvas canvas, Size size) {
  var paint = Paint()..style = PaintingStyle.fill;

  // Треугольник
  var path = Path()
    ..moveTo(size.width / 2, 0)
    ..lineTo(size.width, size.height)
    ..lineTo(0, size.height)
    ..close();
  canvas.drawPath(path, paint..color = Colors.green);

  // Дуга
  canvas.drawArc(
    Rect.fromCircle(center: Offset(150, 150), radius: 60),
    0,            // startAngle
    3.14,         // sweepAngle (π = полукруг)
    true,         // useCenter
    paint..color = Colors.orange,
  );
```

```
  // Скруглённый прямоугольник
  canvas.drawRRect(
    RRect.fromRectAndRadius(
      Rect.fromLTWH(10, 10, 200, 100),
      const Radius.circular(16),
    ),
    paint..color = Colors.purple,
  );

  // Текст на Canvas
  TextPainter(
    text: const TextSpan(text: 'Canvas Text', style: TextStyle(colo
    textDirection: TextDirection.ltr,
  ).layout()..paint(canvas, const Offset(50, 50));
}
```

АДАПТИВНЫЙ ДИЗАЙН: LAYOUTBUILDER И MEDIAQUERY

```
// Полная адаптивная страница
class AdaptivePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        // Мобильная версия
        if (constraints.maxWidth < 600) {
          return Scaffold(
            appBar: AppBar(title: const Text('Мобильная версия')),
            body: _buildMobileLayout(),
            bottomNavigationBar: BottomNavigationBar(
              items: const [
                BottomNavigationBarItem(icon: Icon(Icons.home), labe
                BottomNavigationBarItem(icon: Icon(Icons.person), la
              ],
            ),
          );
```

```
        }
        // Планшет/Desktop версия
        return Scaffold(
          body: Row(
            children: [
              NavigationRail(
                selectedIndex: 0,
                destinations: const [
                  NavigationRailDestination(icon: Icon(Icons.home),
                  NavigationRailDestination(icon: Icon(Icons.person
                ],
              ),
              Expanded(child: _buildDesktopLayout()),
            ],
          ),
        );
      },
    );
  }
}
```

АДАПТИВНЫЕ ВИДЖЕТЫ

```
// Platform-adaptive кнопки
Widget buildButton(BuildContext context) {
  final isIOS = Theme.of(context).platform == TargetPlatform.iOS;

  if (isIOS) {
    return CupertinoButton.filled(
      onPressed: () {},
      child: const Text('Нажми'),
    );
  }
  return ElevatedButton(
    onPressed: () {},
    child: const Text('Нажми'),
  );
}

// Автоматические виджеты
// adaptive widgets:/cupertino (pub.dev)

// Directional-отступы
```

```
const EdgeInsetsDirectional.only(start: 16)  // RTL-safe

// AdaptiveLayout (Material 3)
AdaptiveLayout(
  primaryNavigation: NavigationRail(
    selectedIndex: 0,
    destinations: const [
      NavigationRailDestination(icon: Icon(Icons.home), label: Text
    ],
  ),
  body: _buildBody(),
)
```

WRAP И FLOW: ГИБКИЙ LAYOUT

```
// Wrap - переносит элементы на новую строку
Wrap(
  spacing: 8,
  runSpacing: 8,
  children: [
    Chip(label: Text('Dart')),
    Chip(label: Text('Flutter')),
    Chip(label: Text('Material')),
    Chip(label: Text('Cupertino')),
    Chip(label: Text('Widgets')),
  ],
)

// Flow - позиционирование с трансформациями
Flow(
  delegate: FlowDelegate(),
  children: [
    Icon(Icons.star, size: 40),
    Icon(Icons.star, size: 40),
```

```
    Icon(Icons.star, size: 40),
  ],
)
```

СОВЕТЫ ПО ПРОИЗВОДИТЕЛЬНОСТИ

- Используйте `const` конструкторы где возможно
- Избегайте тяжёлых вычислений в `build()`
- Разделяйте большие виджеты на маленькие
- Используйте `ListView.builder` вместо `ListView`
- Правильно используйте **keys** для виджетов
- Измеряйте производительность через DevTools

ИТОГИ ЛЕКЦИИ 5

- Flutter - кроссплатформенный фреймворк на Dart
- Dart: null-safety, async/await, классы, mixins
- Всё - виджеты: StatelessWidget и StatefulWidget
- Material Design (Android) и Cupertino (iOS)
- Hot Reload для быстрой разработки
- Следующая лекция: навигация и управление состоянием